universität wien

BACHELORARBEIT

ENHANCEMENT OF THE PROCESS MINING VISUALIZATION TOOL: PETRI-NET INTEGRATION, CONVERTERS, TOKEN-GAME SIMULATION, AND HAPPY-PATH VIEWS

Verfasserin ODER Verfasser

Jakob Schütz

angestrebter akademischer Grad

Bachelor of Science (BSc)

Wien, 2026

Studienkennzahl lt. Studienblatt:	UA 033 521
Fachrichtung:	Informatik
Betreuerin / Betreuer:	Ing. Dipl.-Ing. Dr.techn. Marian Lux

Contents

Abstract

1 Motivation

Many organizations manage their daily operations and processes using ERP platforms, ticketing tools, or workflow engines. However, it is a great challenge to convert these low-level event data into a clearly understandable model. In practice, processes often span multiple business areas, involve multiple exceptions, and deviate from official documentation. Process mining aims to make these problems more tangible by using derived process models from event logs and ultimately making the as-is behaviours of the process visible. Nevertheless, the practical value of process mining does not solely depend on discovery algorithms, it also depends on whether the resulting models can be visualized in a consistent and intuitive way, allowing stakeholders to understand them, compare different process variants, and identify opportunities for improvement.

Although there are already numerous process mining tools, there are still issues with accessibility and expandability. Commercial tools usually offer well-designed user interfaces and a wide range of features, but they also usually require expensive licenses, which makes them less attractive for science, teaching, and community-driven development. Academic tools, on the other hand, often have access to many different mining algorithms. However, they are often more complex to use, which can be a hurdle for non-experts. To bridge this gap, an open-source, Python-based process mining visualization tool was developed with a strong focus on usability and extensibility for the interactive exploration of event logs.

Despite the introduction of new features and mining algorithms, the tool still exhibited potential for improvement. In particular, several core components remained incomplete or were implemented only through provisional solutions. For example, the token game was realized using a workaround rather than a dedicated implementation. Furthermore, the Petri net concept had not yet been implemented in a reusable manner, limiting its applicability across different parts of the system.

The motivation of this thesis is therefore to enhance the tool and resolve the identified shortcomings. First, a robust Petri net implementation is developed, which also serves as the foundation for a proper implementation of the token game. Subsequently, the logic of the visualization components is consolidated, reducing the number of supported graph types to BPMN, Directly-Follows, and Petri net graphs. To further decouple visualization logic from mining algorithms, dedicated converter classes are introduced for each graph type. Finally, a new visualization feature is implemented that allows highlighting the most frequent trace within an event log.

2 Related Work

2.1 Existing Tools

In the field of process mining, a wide range of tools exists that differ with respect to functionality, licensing models, and target audiences. These tools can broadly be categorized into commercial and open-source solutions. The following section presents this classification by introducing the most prominent representatives of each category and subsequently discussing the key limitations of existing tools.

2.1.1 Commercial Tools

Commercial process mining tools primarily target business users and offer a high degree of integration with existing IT systems as well as advanced analytical capabilities. A prominent example is Celonis, a largely cloud-based platform that is predominantly used in large organizations. The tool provides the main functionalities such as process discovery, conformance checking, and the enhancement of existing process models. Additionally, Celonis supports a wide range of data mining algorithms, including the Heuristic Miner, the Inductive Miner, and object-oriented process mining approaches. In addition to typical analysis functions, the platform offers customizable dashboards which are tailored to different organizational roles, such as management or specialized departments. Despite this wide range of functions, the applicability of Celonis is limited for smaller organizations and research institutions due to high license and operating costs. [1]

Another widely used commercial process mining tool is Disco, which is primarily aimed at analysts and consultants. Disco is characterized by its ease of use and the ability to deliver quick insights during the exploratory analysis of event logs. The application operates locally as a desktop solution and requires manual data preparation as well as manual data import, typically in CSV format. Unlike company-oriented platforms, Disco focuses on a few analytical functions and supports only one mining algorithm, the Fuzzy Miner. The lack of support for additional algorithms in conjunction with the proprietary architecture significantly limits the tool's adaptability and restricts its applicability in advanced or experimental scenarios. [2]

2.1.2 Open-Source Tools

In contrast to commercial solutions, ProM 6 represents an open-source approach and is therefore widely used in academic research. Its modular framework provides a wide range of plugins for different process mining tasks, including process discovery, conformance checking, and performance analysis. The openness of the platform enables researchers to implement their own custom algorithms and modify existing ones. However, the platform is noticeably more complex to use and requires a deeper understanding of the underlying concepts and process mining principles. Furthermore, ProM is less suitable for industrial environments, since aspects such as scalability, user experience, and integration with operational information systems are only partially addressed. [3]

2.1.3 Critical review of existing tools

Despite the wide variety of available process mining tools, several fundamental weaknesses persist. Commercial solutions typically offer extensive functionality and strong integration with existing IT infrastructures; however, they are often associated with high costs and only limited possibilities for customization. Proprietary algorithms and closed systems decrease transparency and hinder the reproducibility of analysis results. On the other hand, open-source tools such as ProM 6 provide a high degree of flexibility and transparency but are therefore connected with a steep learning curve and relatively complex user interfaces. This comparison reveals a clear gap between practical yet restrictive commercial solutions and flexible but complex research-oriented tools, highlighting the need for an approach that combines usability with openness and extensibility. [4]

3 Algorithms

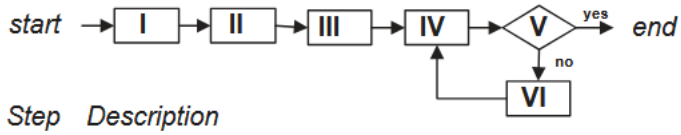
3.1 Genetic Algorithm

A genetic algorithm is a stochastic optimization method based on the principle of biological evolution. The goal is to find a good, but not necessarily global, solution to an optimization problem in a large, often discrete search space. Unlike classical optimization methods, no single solution is incrementally improved. A genetic algorithm works with a population of candidate solutions, called individuals. Each individual represents a possible solution to the problem. The individual's quality is evaluated using a fitness function that returns a numerical score.

Genetic algorithms represent a general optimization framework that can be applied to a variety of domains. In the field of process mining, these principles are adapted in so-called genetic mining algorithms to automatically discover process models from event logs. In this context, individuals represent concrete candidates for process models, such as Petri nets or similar structures, rather than abstract solutions. The evolutionary mechanisms of the genetic algorithm are adapted to generate valid and meaningful process models and improve them step by step. [5] [6]

3.1.1 Process of a Genetic Mining Algorithm

The genetic mining algorithm follows a clearly structured process that is divided into several consecutive steps. Figure 1 shows the main steps of the algorithm, which are explained in more detail below.



Step	*Description*
I	Read event log
II	Calculate dependency relations among activities
III	Build the initial population
IV	Calculate individuals' fitness
V	Stop and return the fittest individuals?
VI	Create next population - use genetic operations

Figure 1: Main steps of the Genetic Algorithm. [5]

I. In the first step, the event log is read. This event log forms the basis for the genetic mining algorithm and contains the observed processes in the form of traces, more on this in Chapter 3.4.
II. Based on the event log, the dependency relationships between activities are calculated in a second step. This includes analysing which activities typically follow one another and in which direction this relationship exists. This information serves as a structural basis for the later model representation and is often stored in the form of a dependency or causality matrix.
III. In the third step, the initial population is built. Each individual represents a possible process model based on the previously calculated dependency relations. The initial population can be randomly generated but is usually limited by heuristic information from the event log to avoid obviously incorrect models.
IV. The fourth step involves calculating the fitness of each individual. Each process model is compared to the event log to assess how well it describes the observed behaviour. The fitness function typically measures the extent to which the traces from the log can be reproduced by the model and how much additional, unobserved behaviour the model allows. Models with higher fitness provide better explanations for the event log.
V. In the fifth step, it is checked whether the termination criterion is met. For example, this can be a maximum number of generations or a time limit.
VI. If the termination criterion is not met, the next population is generated. Genetic operators such as selection, recombination, and mutation are used for this. Fitter individuals are preferentially selected and combined with each other, while mutations deliberately make structural changes to process models to create new variants. The newly formed population then serves as input for the fitness calculation, which starts the evolutionary cycle again. [5]

3.1.2 Fitness and Token Game

The token game, often also referred to as token replay, is based on the formal semantics of Petri nets and related process modelling formalisms. A Petri net consists of Places, Transitions, and Edges (see Chapter 3.2 for details). Tokens are placed on places and represent the current state of the process. A transition can be activated if its input conditions are satisfied. More precisely, a transition is enabled if all required input sets are fulfilled (AND semantics across input sets) and within each input set at least one incoming place contains a token (OR semantics within a set). If that is the case, the transition can be fired which consumes tokens from the selected input places and produces tokens in the corresponding output places. This allows checking whether an observed trace (see Chapter 3.4) can be executed by the model.

During token replay, a single trace of the event log is processed step by step. It starts with a defined start marker. For each event in the trace, the corresponding transition in the model is selected and

fired if it is enabled according to the AND/OR activation rules described above. If a transition cannot be activated, it is interpreted as a discrepancy between the model and the observed behaviour.

After a trace has been fully executed, it is verified whether any tokens remain in the model. Such leftover tokens are an indication that the model still contains open paths or unfulfilled synchronizations after the observed execution. Throughout the replay, the number of consumed tokens, produced tokens, and artificially added tokens is recorded. Based on these quantities, a continuous fitness value is computed that expresses how well the process model fits the trace under consideration. Intuitively, a model achieves a higher fitness if all activities can be successfully parsed according to the AND/OR firing semantics and if no tokens remain after completing the trace.

I. Initialize Petri Net (Start Marking)
II. Select Trace from Event Log
III. Process events sequentially
IV. Check Transition Enablement (AND / OR)
V. Fire Transition
VI. Finish Trace
VII. Compute Fitness Value

The continuous fitness measure is:

$$F(PM, L) = 0.40 \cdot \frac{\text{parsed activities}}{\text{total activities}} + 0.60 \cdot \frac{\text{completed traces}}{\text{total traces}}, \quad F \in [0, 1].$$

3.2 Petri net

A Petri net is a directed graph consisting of places and transitions as it can be seen in figure 3. Places can contain tokens, and transitions fire as soon as their input places have enough tokens. After completing the corresponding mining algorithm, the best candidate model is transformed into a Petri net to make it analysable and visualizable. A workflow-like structure with an explicit start and end is typical: a start node and an end node are connected via dedicated places, so it is clear where tokens are initially generated and where they are collected at the end. The input and output relationships contained in the model are then modelled using intermediate places.

- For each input set of an activity, a Place is created that connects the predecessors with the activity.
- For each output set, a place is created that connects the activity with its successors.
- Self-loops are treated separately by introducing a place that links the activity to itself.

ID	Process instance	ID	Process instance	ID	Process instance
1	X, A, C, D, Y	3	X, A, C, D, Y	5	X, B, C, E, Y
2	X, B, C, E, Y	4	X, B, C, E, Y	6	X, A, C, D, Y

Figure 2 [7]

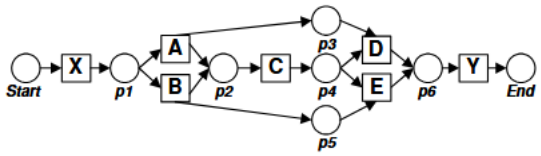


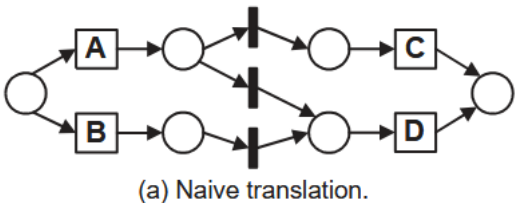
Figure 3 [7]

3.2.1 Silent Transitions

When constructing a Petri net from a causal matrix (see Figure 4), silent transitions are commonly introduced in the literature [5]. These transitions have no counterpart in the event log and are mainly used to correctly represent control flow constructs, such as choices, parallel splits/joins, and certain loop structures, as it can be seen in figure 2.2 with its corresponding causal Matrix in figure 5.

ACTIVITY	*INPUT*	*OUTPUT*
A	$\{\}$	$\{\{C, D\}\}$
B	$\{\}$	$\{\{D\}\}$
C	$\{\{A\}\}$	$\{\}$
D	$\{\{A, B\}\}$	$\{\}$

Figure 4 [5]



(a) Naive translation.

Figure 5 [5]

4 Bibliography

[1] Celonis, “Process intelligence and process mining,,” 2026. [Online]. Available: https://www.celonis.com/. [Accessed 06 01 2026].

[2] Fluxicon, “Process mining and automated process discovery software for professionals,” 2026. [Online]. Available: https://fluxicon.com/disco/. [Accessed 06 01 2026].

[3] P. Tools, “Prom tools,” 2026. [Online]. Available: https://promtools.org/. [Accessed 06 01 2026].

[4] W. M. P. van der Aalst, Process Mining Discovery, Conformance and Enhancement of Business Processes, Springer Verlag Berlin Heidelberg, 2011.

[5] A. K. A. de Medeiros, A. J. Weijters and W. M. P. van der Aalst, “Using genetic algorithms to mine process models: representation, operators and results,” Eindhoven University of Technology, 2004.

[6] A. K. A. de Medeiros, A. J. Weijters and W. M. P. van der Aalst, “Genetic process mining: an experimental evaluation,” *Data Mining and Knowledge Discovery,* vol. 14, no. 3, pp. 245-304, 2007.